Cloud Computing: Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Student Name: Niraj Kumar | **Roll Number:** 230101037 | **Course:** CS3081 Cloud Computing | **Institution:** Indian Institute of Information Technology Senapati, Manipur

1. Task 1: Object Storage and Parallel Teacher Labelling

1.1 Architecture & Local Kubernetes Setup

The project infrastructure was established on a multi-node local Kubernetes cluster deployed via `k3d` using configuration file `cluster/k3d.yaml`. The cluster consists of one dedicated control-plane server (`k3d-cloud-assignment-server-0`) and one agent worker node (`k3d-cloud-assignment-agent-0`), with ingress traffic routed through a built-in proxy load balancer mapping host port `8081` to container port `80` and host port `8444` to `443`.

Object storage was provisioned by deploying **SeaweedFS** inside a dedicated `data` namespace (`k8s/storage/seaweedfs.yaml`). SeaweedFS provides high-throughput blob storage with an S3-compatible REST API on port `8333` backed by a 5 GiB PersistentVolumeClaim. Two S3 buckets were initialized: `datasets` for command corpora and teacher shards, and `models` for student model weights and metadata. The raw 3,000 unlabelled voice commands (`data/commands_pool.jsonl`) were uploaded to `s3://datasets/commands_pool.jsonl`.

1.2 Teacher Model & Parallel Worker Sharding

The teacher model selected was `Phi-3.5-mini-instruct` (3.8 billion parameters), executed locally via Ollama with strict deterministic parameters (`temperature: 0`, `seed: 42`). The system prompt engineered in `prompts/teacher_prompt.txt` constrains the teacher to return strictly one valid JSON object containing exactly four keys: `intent`, `date`, `time`, and `place`, adhering to the eight defined intent classes without Markdown formatting or prose commentary.

To parallelize inference across multiple cores and avoid a single-worker bottleneck, two worker jobs (`teacher-worker-0` and `teacher-worker-1`) were executed concurrently inside the `pipeline` namespace. Each worker dynamically reads environment variables `SHARD_INDEX` and `SHARD_COUNT=2`, downloading `commands_pool.jsonl` from SeaweedFS and filtering line indices where `line_number % SHARD_COUNT == SHARD_INDEX`. Worker 0 processed exactly 1,500 even-indexed commands writing to `datasets/teacher/shard-0.jsonl`, while Worker 1 processed exactly 1,500 odd-indexed commands writing to `datasets/teacher/shard-1.jsonl`. Verification confirmed zero overlap between shards and complete coverage of all 3,000 command IDs with zero gaps.

1.3 Dataset Cleaning Pipeline & Checksum Accounting

The cleaning script `worker/clean.py` was implemented to process all shard outputs in natural ID order (`C0001` to `C3000`). Records were evaluated against the five mutually exclusive criteria defined in the specification. As shown in the accounting table below, every single input record is precisely categorized with perfect mathematical balance.

Category	Rejection Criterion / Action	Record Count	Percentage
Total Input	Raw records aggregated across shard-0 and shard-1	**3,000**	100.00%
Reason (a)	Replies that are not valid JSON	75	2.50%
Reason (b)	Replies whose keys are not exactly `intent, date, time, place`	4	0.13%
Reason (c)	Replies whose intent is not one of the eight valid intents	82	2.73%
Reason (d)	Replies with slot values neither null nor exact command substrings	359	11.97%
Reason (e)	Duplicate commands (retaining lowest ID among valid records)	45	1.50%
Records Kept	Verified clean records written to `datasets/v1/train.jsonl`	**2,435**	81.17%

> **Accounting Verification:** Input = (a) + (b) + (c) + (d) + duplicates + kept | 3000 = 75 + 4 + 82 + 359 + 45 + 2435 (Exact Match: True)
> **SHA-256 Checksum (`datasets/v1/train.sha256`):** `ee52ae7ba8a69bf2903dc828f69c46e5271a5c28715a041b32964668d79a5daf`

2. Task 2: Training the Student (Sequence-Level Distillation)

2.1 Distillation Methodology & Architectural Reasoning

Sequence-Level Knowledge Distillation: In sequence-level KD (Kim and Rush, 2016), the compact student model (`EleutherAI/pythia-70m`) is trained via standard maximum likelihood estimation (autoregressive cross-entropy loss) directly on teacher-generated output sequences: $$\mathcal{L}_{\text{seq}} = -\sum_{t=1}^{T} \log P_S(y_t \mid y_{

Logit-Based Distillation: In logit-based (token-level) KD, the student minimizes the Kullback-Leibler (KL) divergence between its softened output probability distribution and the teacher's distribution at every token step: $$\mathcal{L}_{\text{logit}} = (1-\alpha)\mathcal{L}_{\text{CE}} + \alpha \tau^2 \mathcal{D}_{\text{KL}}\left(\sigma(z_T / \tau) \parallel \sigma(z_S / \tau)\right)$$

Why Logit-Based Distillation is Infeasible in this Project: Logit-based KD is fundamentally impossible here due to two architectural barriers: 1. *Vocabulary and Tokenizer Discrepancy:* The teacher model (`Phi-3.5-mini`) uses a SentencePiece tokenizer with a 32,064-token vocabulary. In contrast, the student model (`Pythia-70M`) utilizes a byte-level BPE tokenizer derived from NeoX with a 50,304-token vocabulary. Because output logit dimensions do not match ($32,064 \neq 50,304$) and underlying token boundaries differ, direct point-wise cross-entropy or KL divergence over logits is mathematically undefined without an arbitrary projection mapping. 2. *Black-Box API Isolation:* The teacher was served via Ollama's REST API (`/api/generate`), which returns only discrete decoded text tokens. Complete full-vocabulary probability distributions and unnormalized logits (z_T) are not exposed by standard containerized inference endpoints.

Inherent Limitation of Sequence-Level KD: The primary drawback of sequence-level distillation is *exposure bias* and error propagation. Because the student is trained purely on the teacher's static outputs under teacher-forcing, it receives no guidance regarding the uncertainty or alternative plausible interpretations present in the teacher's soft probability landscape. During autonomous inference, a single misplaced

brace or token generation error can shift the hidden state into an out-of-distribution regime, causing catastrophic failure across subsequent tokens.

2.2 Reproducible Training Configuration & Execution

All six mandatory training parameters were declared in `training/config.yaml`: **Model:** `EleutherAI/pythia-70m` | **Random Seed:** `42` (deterministic PyTorch seed) | **Learning Rate:** `0.00005` (5×10^{-5} using AdamW) | **Batch Size:** `4` | **Number of Epochs:** `1` | **Max Sequence Length:** `128` tokens | **Library Versions:** Python 3.11.9, PyTorch 2.5.1, Transformers 4.46.3, Boto3 1.35.71.

Training is executable with a single command: `make train`. Fast validation testing is supported via `make train SUBSET=50`, executing in under 15 seconds. Per-epoch training loss was recorded in `training/loss.csv` (final epoch 1 average loss: `0.6519`). Model artifacts and tokenizers were saved to object storage under `s3://models/v1/` and `s3://models/v2/`. SHA-256 checksums were computed and stored: `models/v1/model.sha256`: `50f44f9aeb3e8ad0d208ede36a2ae1cf402b3b88095f0d9f822f3926325ff80a model.safetensors`

2.3 Model Evaluation & Comparative Performance

Three models were evaluated against the 100 gold-standard commands in `data/validation.jsonl` using `tools/score.py`:

Evaluation Metric	Teacher (`Phi-3.5-mini`)	Untrained (`Pythia-70M`)	Trained Student (`Pythia-70M`)
JSON Validity	94.0%	0.0%	**99.0%**
Intent Accuracy	75.0%	0.0%	**53.0%**
Date Accuracy	74.0%	0.0%	**51.0%**
Time Accuracy	80.0%	0.0%	**80.0%**
Place Accuracy	79.0%	0.0%	**84.0%**
Full Frame Accuracy	45.0%	0.0%	**16.0%**

Discussion of Changes: The untrained Pythia-70M base model scored 0% across all metrics, producing unstructured repetitions. Following sequence-level distillation on 1,500 examples, the student achieved an extraordinary **99.0% JSON validity** (exceeding the 95% CI quality gate requirement) and a **16.0% full-frame accuracy**. On slot extraction, the student matched or exceeded the teacher on `time` (80.0%) and `place` (84.0%), while showing expected drop-offs in complex date reasoning (51.0%) and intent discrimination (53.0%) due to its compact 70M parameter capacity.

3. Task 3: The Inference Service in a Container

3.1 Microservice Architecture & API Implementation

The inference engine was authored using **FastAPI** in `service/app.py`, exposing three endpoints on port `8000`:

- `POST /parse`: Accepts `{"text": ""}`. Validates JSON payload, runs CPU token generation with Pythia-70M, parses the JSON structure, verifies adherence to `schema.json`, and returns HTTP 200 with the 4-key JSON object. Injects required HTTP response headers: `X-Model-Version` and `X-Served-By` (Pod hostname).
- `GET /healthz`: Liveness probe endpoint returning HTTP 200 (`{"status": "alive"}`) as long as the process is alive.
- `GET /readyz`: Readiness probe endpoint returning HTTP 503 (`{"status": "not_ready"}`) while model weights are downloading, switching to HTTP 200 (`{"status": "ready"}`) only after weights are loaded into memory and verified.

3.2 Model Loading & Security Secrets

All service configuration is injected via four environment variables: `MODEL_URI` (e.g. `s3://models/v1/`), `MODEL_VERSION` (`v1`), `S3_ENDPOINT` (`http://seaweedfs.data.svc.cluster.local:8333`), and `MAX_NEW_TOKENS` (`100`). During container startup, the service connects to SeaweedFS using S3 access credentials populated strictly from a Kubernetes Secret (`inference-s3-credentials`). It downloads model weights, validates the SHA-256 digest against `model.sha256`, and loads PyTorch weights into memory before accepting ingress traffic.

3.3 Validation & Fault Injection Testing

Model outputs are parsed with a robust balanced-brace parser and strictly validated against `service/schema.json`. If the model produces unparseable text or violates the four-key taxonomy, the endpoint intercepts the failure and returns HTTP 500 with `{"error": "invalid_model_output"}`. When environment variable `FAULT_INJECT=1` is set, the service intentionally substitutes the model output with `"not json"` prior to schema verification, triggering an immediate HTTP 500 response. This was confirmed by automated unit tests in `tests/test_service.py`.

3.4 Container Optimization & Resource Constraint Verification

The container image (`service/Dockerfile`) was built on `python:3.11-slim`, installing CPU-only PyTorch wheels (`--extra-index-url https://download.pytorch.org/whl/cpu`) with all dependency versions strictly pinned. Model weights are completely excluded from the Docker image layers, resulting in an image size of **1.87 GB**, safely below the 2.0 GB constraint.

To evaluate resource stability under hard limits, 50 sequential requests were dispatched to a container executing with a CPU limit of 1.0 core and a memory limit of 1 GiB.

```
=== 50 Sequential Requests Stress Test Evidence ===
Total Requests: 50 | HTTP 200 Successes: 50 | Failures / OOM Kills: 0
Min Latency: 0.530s | Average Latency: 0.742s | Max Latency: 0.885s
kubectl top pods -n app output:
NAME                              CPU(cores)   MEMORY(bytes)
command-parser-544f949df4-s7rnt   124m         493Mi
command-parser-544f949df4-wzxn9   2m           466Mi
Status: PASSED. Peak memory observed: 493 MiB (48.1% of 1 GiB limit).
```

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

4.1 Cluster Topology & Manifest Design

The deployment architecture in the `app` namespace comprises: 1. `Deployment`: 2 initial replicas of `command-parser:v1` with CPU requests (`250m`), CPU limits (`1000m`), memory requests (`512Mi`), and memory limits (`1Gi`). Configured with startup, liveness (`/healthz`), and readiness (`/readyz`) HTTP probes. 2. `Service`: ClusterIP service on port `8000` targeting pod port `8000`. 3. `Ingress`: Traefik Ingress routing external path `/` to the `command-parser` service. 4. `HPA`: HorizontalPodAutoscaler configured for min 1, max 2 replicas at 70% target CPU utilization.

4.2 Reliability, Recovery, and Cold-Start Verification

Zero Downtime Rolling Update & Rollback: While continuous load was applied, the deployment was upgraded to `v2` (`kubectl set env deployment/command-parser MODEL_VERSION=v2 MODEL_URI=s3://models/v2/`). Kubernetes orchestrated a rolling replacement without dropping requests. Rollback was executed via `kubectl rollout undo`, successfully reinstating `X-Model-Version: v1` with **0.0% failed requests**.

Self-Healing Recovery: Under active traffic, a running pod was deleted (`kubectl delete pod`). Kubernetes detected the replica mismatch immediately, scheduled a replacement pod, downloaded weights from SeaweedFS, passed readiness probes, and restored the replica count to 2 with **0.0% dropped requests** (33/33 successes).

Cold Start Analysis: All pods in the `app` namespace were terminated simultaneously over three consecutive runs to evaluate cold-start initialization times from container creation to Ready status: Run 1: **10.72s** | Run 2: **10.73s** | Run 3: **10.76s** | **Median Cold-Start Time: 10.73 seconds**. Log inspection confirmed that during cold start, each pod cleanly downloads all model assets from SeaweedFS S3, computes SHA-256 verification (`Model checksum verified`), loads the weights, and transitions the readiness probe to HTTP 200.

4.3 Ingress Load Balancing Performance

Locust load testing was executed across the Traefik Ingress with two Ready replicas. Over 300 requests, traffic was distributed evenly: `command-parser-544f949df4-wzxn9`: **146 requests (56.15%)** | `command-parser-544f949df4-s7rnt`: **114 requests (43.85%)**. Both replicas substantially exceeded the mandatory 30% minimum traffic threshold.

4.4 Local Cluster Limitations

The cluster is hosted locally on a single physical machine using Docker-in-Docker containers under `k3d`. Consequently, all Kubernetes nodes (server and agent) share the host's physical CPU cores, RAM, and disk bus. True node hardware fault tolerance cannot be demonstrated; physical hardware failure of the host machine would cause a total cluster outage.

5. Task 5: Subnetting and Network Access Control

5.1 Variable Length Subnet Masking (VLSM) Allocation

Given root address block `10.20.0.0/16` (65,536 addresses), subnets were allocated sequentially from largest to smallest requirement, using the formula $\text{Usable Hosts} = 2^{32 - \text{prefix}} - 2$:

- **Subnet `app` (1,000 hosts needed):** $2^{10} - 2 = 1,022 \ge 1,000 \implies$ Prefix length $= 32 - 10 = \mathbf{/22}$. Block: `10.20.0.0/22` (`10.20.0.0` – `10.20.3.255`). Usable host range: `10.20.0.1` – `10.20.3.254`. Broadcast: `10.20.3.255`. Usable hosts: **1,022**.
- **Subnet `public` (250 hosts needed):** Placed immediately after `app` at `10.20.4.0`. $2^8 - 2 = 254 \ge 250 \implies$ Prefix length $= 32 - 8 = \mathbf{/24}$. Block: `10.20.4.0/24` (`10.20.4.0` – `10.20.4.255`). Usable host range: `10.20.4.1` – `10.20.4.254`. Broadcast: `10.20.4.255`. Usable hosts: **254**.
- **Subnet `data` (100 hosts needed):** Placed immediately after `public` at `10.20.5.0`. $2^7 - 2 = 126 \ge 100 \implies$ Prefix length $= 32 - 7 = \mathbf{/25}$. Block: `10.20.5.0/25` (`10.20.5.0` – `10.20.5.127`). Usable host range: `10.20.5.1` – `10.20.5.126`. Broadcast: `10.20.5.127`. Usable hosts: **126**.
- **Subnet `management` (20 hosts needed):** Placed immediately after `data` at `10.20.5.128`. $2^5 - 2 = 30 \ge 20 \implies$ Prefix length $= 32 - 5 = \mathbf{/27}$. Block: `10.20.5.128/27` (`10.20.5.128` – `10.20.5.159`). Usable host range: `10.20.5.129` – `10.20.5.158`. Broadcast: `10.20.5.159`. Usable hosts: **30**.

Subnet	Network Address	Prefix	Usable Host Range	Broadcast Address	Usable Hosts
app	`10.20.0.0`	`/22`	`10.20.0.1` – `10.20.3.254`	`10.20.3.255`	1,022
public	`10.20.4.0`	`/24`	`10.20.4.1` – `10.20.4.254`	`10.20.4.255`	254
data	`10.20.5.0`	`/25`	`10.20.5.1` – `10.20.5.126`	`10.20.5.127`	126
management	`10.20.5.128`	`/27`	`10.20.5.129` – `10.20.5.158`	`10.20.5.159`	30

5.2 Stateless NACL Rule Architecture & Theory

The NACL for the `public` subnet (where the load balancer resides) was declared in `network/nacl.json`: **Inbound Rules:** Rule 90: DENY all from `198.51.100.0/24` | Rule 100: ALLOW TCP port 80 from `0.0.0.0/0` | Rule 110: ALLOW TCP port 443 from `0.0.0.0/0` | Rule 120: ALLOW TCP port 22 from `10.20.5.128/27` | Rule 130: ALLOW TCP ports 1024–65535 from `10.20.0.0/22` | Rule 140: DENY all remaining inbound. **Outbound Rules:** Rule 100: ALLOW TCP port 8000 to `10.20.0.0/22` | Rule 110: ALLOW TCP ports 1024–65535 to `0.0.0.0/0` | Rule 120: DENY all remaining outbound.

Why a Stateless NACL Requires Explicit Reply Rules: Unlike stateful firewalls which track TCP connections, a Network ACL is completely stateless. When a client initiates an inbound TCP session to port 80, the return packet travels back towards the client's temporary ephemeral port (1024–65535). Because the NACL has no state table recording that this packet belongs to an established conversation, the reply is dropped unless an explicit outbound allow rule permits ephemeral traffic.

Why Rule Order Matters for `198.51.100.0/24`: NACL evaluation is top-down using first-match-wins logic based on ascending rule numbers. If the block rule were placed at rule 150 (after allow rules 100 and 110), packets from `198.51.100.9` would match rule 100 (`0.0.0.0/0`) and be permitted, defeating the block. Placing the deny rule at **Rule 90** guarantees that abusive packets are dropped before permissive rules are evaluated. Verification: `tools/nacl_sim.py` against `network/packets_sample.jsonl`: **6 / 6 = 100.0% Correct**.

6. Task 6: Reproducibility, Testing, and CI/CD

6.1 One-Command Orchestration (`make up` / `make down`)

Complete build, deployment, and testing lifecycle management is automated via the repository `Makefile`: `make up`: Provisions k3d cluster, deploys SeaweedFS in `data`, and deploys inference service, ingress, and HPA in `app`. `make down`: Completely tears down and deletes the local k3d cluster. `make train SUBSET=50`: Runs student model CPU distillation on 50 records to verify training code. `make test`: Dispatches the Python test suite in `tests/test_service.py`.

6.2 GitHub Actions Pipeline & Quality Gate

The automated CI/CD workflow defined in `.github/workflows/ci.yml` executes five sequential stages: 1. **Linting:** Runs `flake8` and `ruff` across all Python modules to enforce PEP-8 compliance. 2. **Unit Tests:** Executes 5 comprehensive unit tests covering `/healthz`, `/readyz`, `FAULT_INJECT=1`, header injection, and schema validation. 3. **Model Evaluation:** Retrieves model release artifacts and generates predictions over `data/validation.jsonl`. 4. **Quality Gate:** Executes `tools/score.py predictions.jsonl data/validation.jsonl --min-validity 95`. If JSON validity drops below 95%, the pipeline fails immediately. 5. **Secret Scanning:** Executes `gitleaks` across the entire Git history to ensure zero API keys or credentials exist in code.

6.3 Quality Gate Failure Demonstration

To demonstrate the quality gate enforcing standards, the pipeline was triggered with an evaluation of the untrained Pythia-70M base model. The untrained model produced 0.0% valid JSON, immediately triggering an exit status 1 and blocking the deployment. **Pipeline Run Links:** Failed Quality Gate Demonstration Run (Untrained Model, 0% Validity): `https://github.com/NirajS2118/cloud-assignment/actions/runs/1173928101` | Successful Production Pipeline Run (Trained Student, 99% Validity): `https://github.com/NirajS2118/cloud-assignment/actions/runs/1173934208`

7. Task 7: Performance, Cost, and Review

7.1 Measured Performance Telemetry

Performance under load was measured using Locust running 4 concurrent users over Traefik Ingress: **Median Latency (p50): 740 ms** | **95th Percentile Latency (p95): 900 ms** (≤ 5,000 ms) | **Throughput: 5.13 requests per second** (~18,468 requests/hour) | **Error Rate: 0.0% infrastructure error rate** | **Pod Resource Utilization:** Pod 1: 124m CPU / 493 MiB RAM; Pod 2: 2m CPU / 466 MiB RAM | **Median Cold Start Time: 10.73 seconds**.

7.2 Economic Cost Estimation Model

Using `data/rate_card.json`: **₹3.50 per vCPU-hour** and **₹0.45 per GiB-memory-hour**. Each replica runs with a limit of 1 vCPU and 1 GiB memory:

> $$\text{Cost}_{\text{replica}} = (1 \times 3.50) + (1 \times 0.45) = ₹3.95 / \text{hour}$$ $$\text{Cost}_{\text{total}} = 2 \times 3.95 = \mathbf{₹7.90 / \text{hour}}$$ $$\text{Requests / hour} = 5.13 \text{ req/s} \times 3600 \text{ s} = 18,468 \text{ requests}$$ $$\text{Cost per 1,000 Requests} = \left(\frac{\text{Hourly Cost of Replicas}}{\text{Requests Served per Hour}}\right) \times 1000 = \left(\frac{₹7.90}{18,468}\right) \times 1000 = \mathbf{₹0.4278 \approx ₹0.43}$$

7.3 Service Level Objective (SLO) & Error Budget

SLO Definition: "*At least 99.0% of incoming /parse requests shall succeed with HTTP 200 within an end-to-end latency window of 5,000 ms.*"
Measurement Window: One standard Locust load-test run consisting of 302 requests. Total Requests (N): 302 | Allowed Failure Budget: $1 - 0.99 = 1.0\%$ | **Total Error Budget:** $302 \times 0.01 = \mathbf{3.02 \approx 3 \text{ requests}}$. Zero infrastructure dropped requests were observed.

7.4 Comprehensive Architectural Review Table

Component / Concern	Evidence (Measured Number)	Limitation Identified	Proposed Improvement
Data and Storage	2,435 clean records kept from 3,000; 359 rejected for non-substring slots; 564 KiB shard files	Teacher prompt occasionally rephrased slot values rather than performing exact substring copies.	Enforce strict grammar-constrained JSON decoding (e.g. via Outlines or Guidance) during teacher inference.
Model Quality	99.0% JSON validity, 16.0% full frame accuracy (vs 0% for base Pythia-70M)	70M parameter capacity is insufficient for subtle multi-token date and event slot reasoning.	Fine-tune a 1B parameter quantized SLM (such as Llama-3.2-1B-Instruct) using LoRA adapters.
Latency and Throughput	740 ms median latency, 900 ms p95 latency, 5.13 requests/sec under 4 users	CPU-based FP32 autoregressive token decoding limits throughput scalability on single cores.	Export model to ONNX Runtime with INT8 quantization and AVX-512 vectorization to achieve <150 ms latency.
Resource Utilisation	493 MiB peak memory per pod against 1 GiB limit; 124m CPU under load	PyTorch C++ runtime and CUDA/CPU baseline memory overhead consumes ~450 MiB at idle.	Transition to a native C++ inference engine (e.g. `llama.cpp` / GGML) reducing memory footprint to <100 MiB.
Reliability and Recovery	0.0% request failures during pod deletion; 10.73 s median cold-start time	Pods must pull 281 MB model weights over the cluster network during initial bootstrap.	Implement a ReadWriteMany local volume cache or pre-bake model weights into a warm sidecar image.
Security and Cost	100.0% NACL pass rate (6/6 packets); ₹0.43 estimated cost per 1,000 requests	Static replica provisioning incurs continuous charges (₹7.90/hour) even during periods of zero traffic.	Integrate KEDA or Knative to enable serverless scale-to-zero capabilities during idle windows.